ENGINEERING CAPSTONE PROJECT

ShopWave Microservices

Deploy & Manage a Production-Grade Application

Terraform • Kubernetes • Jenkins • Argo CD • Prometheus • Grafana

5 Days	**7 Services**	**3 Databases**	**Full Stack**
Duration	Microservices	Backends	DevOps Pipeline

Project Overview

This 5-day project challenges engineering students to build and operate a complete, production-style microservices deployment pipeline from scratch. You will work with industry-standard DevOps tooling used by real engineering teams at scale.

Project Goal

Take the ShopWave application and build a complete end-to-end production deployment pipeline covering infrastructure provisioning, containerization, CI/CD automation, GitOps delivery, and observability monitoring.

Application Architecture

Implement the following pipeline flow:

```
Terraform → AWS Infra / EKS / Networking → Docker Images → Jenkins
Pipeline → Kubernetes Deployment → Argo CD Sync → Prometheus + Grafana
Monitoring
```

Application Services

Service	Port	Type
`frontend`	3000	User-facing UI
`api-gateway`	80	API entry point
`auth-service`	3001	Authentication & authorization
`product-service`	3002	Product catalog management
`order-service`	3003	Order processing
`payment-service`	3004	Payment handling
`notification-service`	3005	Notifications & alerts

Stateful / Backend Dependencies

- PostgreSQL — relational data store
- MongoDB — document-based storage
- Redis — caching and session management

DAY 1 Infrastructure Provisioning with Terraform

Objective

Students will provision the complete cloud infrastructure required to host the ShopWave application on AWS. By the end of this day, a fully functional Kubernetes cluster should be reachable and ready for workloads.

Tasks

- Create a Terraform project with reusable, modular structure
- Provision core networking components:
  - VPC with DNS support
  - 2 Public Subnets
  - 2 Private Subnets
  - Internet Gateway
  - NAT Gateway
  - Route Tables (public + private)
  - Security Groups
- Provision an Amazon EKS cluster
- Create managed node group(s) with appropriate instance types
- Configure Terraform providers and define outputs
- Store Terraform state remotely using S3 backend with DynamoDB locking

Required Terraform Files

☐ Required Files / Deliverables	
provider.tf	— AWS provider configuration
variables.tf	— Input variable declarations
terraform.tfvars	— Variable values (region, cluster name, etc.)
main.tf	— Core resource definitions
outputs.tf	— Output values (EKS name, VPC ID, subnet IDs)
— OR use a modular layout —	
modules/vpc/	— VPC, subnets, gateways, route tables
modules/eks/	— EKS cluster and node groups

Expected Terraform Outputs

- EKS cluster name and ARN

- VPC ID
- Public and private subnet IDs
- Node group details (instance type, desired/min/max size)
- kubeconfig access working via aws eks update-kubeconfig

✔ Validation Commands

```
terraform init
terraform plan
terraform apply
aws eks update-kubeconfig --region <region> --name <cluster-name>
kubectl get nodes
```

 End-of-Day Deliverable: A working EKS Kubernetes cluster on AWS, fully accessible via kubectl.

DAY 2 Containerization & Kubernetes Manifests

Objective

Students will containerize all microservices and write complete Kubernetes YAML manifests to deploy the full application stack manually. By end of day, the app should be accessible through an Ingress or load balancer.

Tasks

- Build Docker images for all 7 application services
- Push images to Docker Hub or AWS ECR
- Create Kubernetes Namespace for ShopWave
- Write Deployments and Services for all application pods
- Create ConfigMaps for non-sensitive configuration
- Create Secrets for credentials and sensitive data
- Write Ingress manifest to expose the frontend
- Configure Horizontal Pod Autoscaler (HPA) for selected services
- Deploy stateful backends: PostgreSQL, MongoDB, Redis (using StatefulSets or Deployments)
- Define environment variables, readiness probes, liveness probes, and resource limits for all services

Required Kubernetes Files

☐ Required Files / Deliverables	
`namespace.yaml`	— shopwave namespace
`configmap.yaml`	— shared app configuration
`secret.yaml`	— DB passwords, API keys
`frontend-deployment.yaml`	`product-deployment.yaml`
`frontend-service.yaml`	`product-service.yaml`
`api-gateway-deployment.yaml`	`order-deployment.yaml`
`api-gateway-service.yaml`	`order-service.yaml`
`auth-deployment.yaml`	`payment-deployment.yaml`
`auth-service.yaml`	`payment-service.yaml`
	`notification-deployment.yaml`
	`notification-service.yaml`

```
postgres-statefulset.yaml          redis-deployment.yaml
postgres-service.yaml              redis-service.yaml
mongodb-statefulset.yaml
mongodb-service.yaml


ingress.yaml                       — frontend/API gateway routing
hpa.yaml                           — autoscaling policy
```

Critical Pod Requirements

- Inter-service communication via Kubernetes Service DNS names
- All services must declare CPU/memory requests and limits
- Readiness probes ensure traffic is only routed to healthy pods
- Liveness probes allow Kubernetes to restart unhealthy pods
- Environment variables injected via ConfigMap and Secret references

✔ Validation Commands

```
kubectl apply -f .
kubectl get pods -n shopwave
kubectl get svc -n shopwave
kubectl get ingress -n shopwave
# Frontend should be accessible via Ingress or LoadBalancer IP
```

 End-of-Day Deliverable: Full ShopWave application running on Kubernetes using manually applied YAML manifests.

DAY 3 Jenkins CI/CD Pipeline

Objective

Students will automate the entire build and deployment workflow using Jenkins. A single push to the repository should trigger the full pipeline — from building Docker images to deploying updated manifests to the cluster.

Tasks

- Install and configure Jenkins (EC2, Docker, or in-cluster)
- Connect Jenkins to the Git repository
- Configure required tools in Jenkins:
  - Git, Docker, kubectl, aws cli
- Create Jenkins credentials for:
  - Git repository (SSH key or token)
  - Docker Hub or ECR (username/password)
  - Kubernetes or AWS access (kubeconfig or IAM role)
- Write a complete Jenkinsfile with all required stages

Required Pipeline Stages

#	Stage	Description
1	**Checkout Code**	Pull latest source code from Git repository
2	**Build Images**	docker build for all 7 services
3	**Tag Images**	Apply version/commit tag to each image
4	**Push to Registry**	Push tagged images to Docker Hub or ECR
5	**Update Manifests**	Replace image tags in Kubernetes YAML files
6	**Deploy to Cluster**	kubectl apply or update GitOps repo for Argo CD
7	**Verify Rollout**	kubectl rollout status for all deployments
8*	**SonarQube Scan**	Optional: static code quality analysis
9*	**Trivy Scan**	Optional: container image vulnerability scan

* Optional bonus stages for advanced students

✔ Validation Commands

```
# Verify in Jenkins UI:
  - Pipeline run completes with all stages GREEN
  - Images available in Docker Hub or ECR registry
```

```
# Verify on cluster:
kubectl get pods -n shopwave
kubectl rollout status deployment/<service> -n shopwave
```

 End-of-Day Deliverable: One-click CI/CD pipeline — a Git push triggers automated build, push, and deploy to Kubernetes.

DAY 4 Argo CD GitOps Integration

Objective

Students will implement GitOps-based continuous deployment using Argo CD. The cluster state will be driven entirely from Git, and any manual drift will be automatically corrected.

Tasks

- Install Argo CD in the Kubernetes cluster
- Expose the Argo CD UI (LoadBalancer or port-forward)
- Create a dedicated GitOps repository (or GitOps folder within the repo)
- Store all Kubernetes manifests in Git under a tracked path
- Create an Argo CD Application resource for ShopWave
- Enable auto-sync with self-healing enabled
- Demonstrate drift detection — manually change a resource and watch Argo CD revert it

Recommended Repository Structure

Required Files / Deliverables
`app-source/  — Application source code`
`  services/`
`  Dockerfile`
`gitops-repo/  — Deployment manifests (Argo CD watches this)`
`  k8s/`
`    namespace.yaml`
`    frontend-deployment.yaml`
`    api-gateway-deployment.yaml`
`    ... (all service manifests)`
`  argocd-app.yaml  — Argo CD Application definition`

Required Deliverables

- argocd-app.yaml — Application resource with syncPolicy configured
- Git repository (or folder) containing all deployment manifests
- Screenshot or proof of Argo CD showing Healthy + Synced status
- Demonstration of self-healing: delete a pod or change a replica count manually, then show Argo CD reverting it

✔ Validation Commands

```
kubectl get pods -n argocd
# In Argo CD UI verify:
  - Application Status = Healthy
  - Sync Status     = Synced

# Drift correction demo:
kubectl delete pod <pod-name> -n shopwave     # Argo CD will recreate it
kubectl scale deployment frontend --replicas=0 -n shopwave  # Argo CD reverts
```

 End-of-Day Deliverable: GitOps-driven continuous deployment — Argo CD keeps cluster state in sync with Git automatically.

DAY 5 Monitoring with Prometheus & Grafana + Final Demo

Objective

Students will instrument the cluster with a full observability stack using Prometheus and Grafana. They will build dashboards and finish with a complete end-to-end live demonstration of the entire project.

Monitoring Tasks

- Install Prometheus using the kube-prometheus-stack Helm chart
- Install Grafana (included in kube-prometheus-stack)
- Configure scraping for the shopwave namespace
- Access Grafana dashboard and verify Prometheus data source
- Create at least one custom dashboard with meaningful panels
- Configure at least one alert rule (e.g., pod restart threshold)

Recommended Installation

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
helm install kube-prometheus-stack prometheus-community/kube-prometheus-stack \
  --namespace monitoring --create-namespace
```

This single chart installs: Prometheus, Grafana, Alertmanager, Node Exporter, and kube-state-metrics.

Minimum Dashboard Requirements

Metric	What to Show
CPU Usage	Per-node and per-pod CPU utilization %
Memory Usage	RAM consumption across pods and nodes
Pod Restarts	Restart count per service over time
Node Metrics	Node health, disk, network I/O
Deployment Health	Available vs. desired replicas

Final Demo Checklist

✓	Demonstration Item
☐	Infrastructure created with Terraform (show terraform apply output or AWS Console)
☐	EKS cluster running with kubectl get nodes
☐	All ShopWave pods running in shopwave namespace

☐	Jenkins pipeline executed successfully (Green build)
☐	Docker images available in registry with correct tags
☐	Argo CD Application showing Healthy + Synced
☐	Argo CD self-healing demo (manual drift corrected automatically)
☐	Prometheus targets all Healthy
☐	Grafana dashboard showing live cluster metrics
☐	Frontend application reachable and functional via browser

✔ Validation Commands

```
kubectl get pods -n monitoring
kubectl get svc -n monitoring
# Access Grafana: kubectl port-forward svc/kube-prometheus-stack-grafana 3000:80 -n monitoring
# Default credentials: admin / prom-operator
```

End-of-Day Deliverable: Complete live demo with all 5 days integrated: Infra → K8s → CI/CD → GitOps → Monitoring.

5-Day Schedule Summary

Day	Focus	Key Tool(s)	Deliverable
1	**Infrastructure Provisioning**	`Terraform + AWS EKS`	Working Kubernetes cluster
2	**Containerization & K8s Manifests**	`Docker + kubectl`	App running on Kubernetes
3	**CI/CD Automation**	`Jenkins + Jenkinsfile`	Automated build & deploy pipeline
4	**GitOps Deployment**	`Argo CD + Git`	Self-healing GitOps delivery
5	**Monitoring + Final Demo**	`Prometheus + Grafana`	Full demo of complete system

Good luck! Build something production-worthy.

Infrastructure • Containers • CI/CD • GitOps • Observability